

Fine-tuning a masked language model

In many NLP applications, if you want to use a Transformer model, you can take a pretrained model from the Hugging Face Hub and directly fine-tune it on your own task-specific data.

If the corpus used for pretraining and the corpus used for fine-tuning are not very different, transfer learning usually gives good results.

However, in some cases, before training a task-specific head, you first need to fine-tune the language model on your own data.

For example, if your dataset contains legal contracts or scientific articles, then a general Transformer model like BERT may treat the domain-specific words in your corpus as rare tokens. As a result, the performance may not be very good.

To reduce this problem, if you first fine-tune the language model on your own domain-specific data, the performance on downstream tasks can improve a lot. The good thing is that usually you only need to do this once.

This process of fine-tuning a pretrained language model on your own domain data is usually called **domain adaptation**.

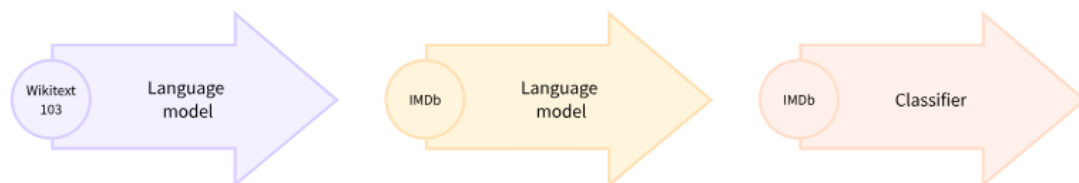
It became popular in 2018 through **ULMFiT**. ULMFiT was one of the first neural architectures in NLP to make transfer learning work effectively, and it was based on LSTMs.

In this section, we will do something similar, but instead of using an LSTM, we will use a Transformer.

By the end of this section, you will have a masked language model on the Hugging Face Hub that can autocomplete sentences.

Let's begin!

If the terms “masked language modeling” and “pretrained model” feel unfamiliar, you can check Chapter 1 first.



Picking a pretrained model for masked language modeling

To begin, we need to choose a pretrained model that is suitable for masked language modeling. On the Hugging Face Hub, if you apply the “Fill-Mask” filter, you can see a list of such models.

Although the BERT and RoBERTa families are the most downloaded, here we will use **DistilBERT**, because it can be trained much faster and usually gives little to no loss in downstream performance.

This model was trained using a technique called **knowledge distillation**. In this method, a large “teacher model” like BERT is used to guide the training of a smaller “student model.” As a result, the student model has fewer parameters, but still keeps much of the performance.

Now let’s download DistilBERT using the `AutoModelForMaskedLM` class:

```
from transformers import AutoModelForMaskedLM

model_checkpoint = "distilbert-base-uncased"
model = AutoModelForMaskedLM.from_pretrained(model_checkpoint)
```

Now we can check how many parameters the model has using the `num_parameters()` method:

```
distilbert_num_parameters = model.num_parameters() / 1_000_000
print(f">>> DistilBERT number of parameters: {round(distilbert_num_parameters)}M")
print(f">>> BERT number of parameters: 110M")
```

Output:

```
'>>> DistilBERT number of parameters: 67M'
'>>> BERT number of parameters: 110M'
```

So DistilBERT has about **67 million** parameters, whereas the BERT base model has **110 million**.

That means DistilBERT is about 2 times smaller, and training is also roughly 2 times faster.

Now let’s see what token the model thinks is most likely to fit in the `[MASK]` position in a short text:

```
text = "This is a great [MASK]."
```

As humans, we may imagine many possibilities for [MASK], such as “day,” “ride,” or “painting.”

But the prediction of a pretrained model depends on the corpus it was trained on, because the model learns the statistical patterns in the data.

Like BERT, DistilBERT was pretrained on English Wikipedia and BookCorpus, so we expect the predictions for [MASK] to reflect those domains.

To predict the mask, we need a tokenizer, so let’s download that too:

```
from transformers import AutoTokenizer
```

```
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)
```

Now, using the tokenizer and the model, we can pass the text to the model, extract the logits, and see the top 5 candidate predictions:

```
import torch
```

```
inputs = tokenizer(text, return_tensors="pt")
token_logits = model(**inputs).logits
# Find the location of [MASK] and extract its logits
mask_token_index = torch.where(inputs["input_ids"] == tokenizer.mask_token_id)[1]
mask_token_logits = token_logits[0, mask_token_index, :]
# Pick the [MASK] candidates with the highest logits
top_5_tokens = torch.topk(mask_token_logits, 5, dim=1).indices[0].tolist()
```

```
for token in top_5_tokens:
```

```
    print(f">>> {text.replace(tokenizer.mask_token, tokenizer.decode([token]))}")
```

Output:

```
'>>> This is a great deal.'
'>>> This is a great success.'
'>>> This is a great adventure.'
'>>> This is a great idea.'
'>>> This is a great feat.'
```

From this output, we can see that the model predicts common everyday English words, which is natural because it was trained on general-domain data like English Wikipedia.

Next, we will see how to change the domain to something more niche, such as highly polarized movie reviews.

Simple Explanation

Now let me explain the whole thing in a very easy way.

Suppose a student has already read a lot of books. So they already have a good understanding of general English.

That is a **pretrained model**.

Now if you want that student to understand medical, legal, or scientific writing, they may not recognize many words properly.

So first, you make the student read books or data from that specific domain.

That is called **domain adaptation**.

Then if you give that student a specific task, such as:

- sentiment analysis
- classification
- question answering

they will perform much better.

What is the main idea here?

In this part, the author is saying:

Instead of directly starting task training, first adapt the model to your own domain-specific text.

Because:

- a general BERT model does not know all kinds of domain-specific words equally well
- it may treat special domain words as rare or unfamiliar
- so if you fine-tune it first on domain data, the later results become better

What is a Masked Language Model?

Here, one word in a sentence is hidden, for example:

This is a great [MASK].

The model's job is:

- to guess which word is most likely to appear in place of [MASK]

For example, the model predicted:

- deal
- success
- adventure
- idea
- feat

That means the model is doing sentence completion.

Why was DistilBERT used?

Because:

- it is smaller than BERT
- training is faster
- performance is almost similar

In numbers:

- BERT = 110M parameters
- DistilBERT = 67M parameters

So it has smaller size, less computation cost, and higher speed.

Why do we need a tokenizer?

The model does not understand text directly.
It understands numbers.

So the tokenizer's job is:

- to split text into tokens
- to convert tokens into IDs
- to prepare the format that the model can understand

Example:

"This is a great [MASK]."

The tokenizer breaks this into pieces and prepares it as model input.

What are logits?

When the model makes a prediction, it first gives raw scores.

These raw scores are called **logits**.

Then:

- the higher the score
- the more likely that token is

In this code, the top 5 highest-score tokens were extracted.

What is the code doing in simple terms?

1) Loading the model

```
model = AutoModelForMaskedLM.from_pretrained(model_checkpoint)
```

Here, a pretrained model suitable for masked language modeling is being loaded.

2) Loading the tokenizer

```
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)
```

Here, the tool that converts text into a format the model can understand is being loaded.

3) Tokenizing the text

```
inputs = tokenizer(text, return_tensors="pt")
```

This converts the sentence into token IDs.

4) Getting the model's prediction

```
token_logits = model(**inputs).logits
```

The model gives scores for all possible tokens.

5) Finding where [MASK] is

```
mask_token_index = torch.where(inputs["input_ids"] == tokenizer.mask_token_id)[1]
```

6) Taking only the scores for the [MASK] position

```
mask_token_logits = token_logits[0, mask_token_index, :]
```

7) Extracting the top 5 predictions

```
top_5_tokens = torch.topk(mask_token_logits, 5, dim=1).indices[0].tolist()
```

8) Converting tokens into words and printing them

for token in top_5_tokens:

```
    print(f">>> {text.replace(tokenizer.mask_token, tokenizer.decode([token]))}")
```

Dataset

To demonstrate domain adaptation, we will use the **Large Movie Review Dataset (IMDb)**. This is a famous corpus of movie reviews that is widely used to benchmark sentiment analysis models.

By fine-tuning DistilBERT on this corpus, we expect the language model to adapt its vocabulary from the factual language of Wikipedia to the more subjective language of movie reviews.

We can load the data from the Hugging Face Hub using the `load_dataset()` function.

```
from datasets import load_dataset

imdb_dataset = load_dataset("imdb")
imdb_dataset
```

Output:

```
DatasetDict({
  train: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  test: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  unsupervised: Dataset({
    features: ['text', 'label'],
    num_rows: 50000
  })
})
```

Here we can see:

- the `train` split contains 25,000 reviews
- the `test` split contains 25,000 reviews

- the `unsupervised` split contains 50,000 reviews, which are unlabeled

Now we can look at a few samples to understand what kind of text we are dealing with. Like in previous chapters, we use `Dataset.shuffle()` and `Dataset.select()` together to create a random sample.

```
sample = imdb_dataset["train"].shuffle(seed=42).select(range(3))
```

for row in sample:

```
    print(f"\n>>> Review: {row['text']}")
```

```
    print(f">>> Label: {row['label']}")
```

From the output, we can see some movie reviews and their labels. From this we understand:

- `0` means a negative review
- `1` means a positive review

Although we do not need the labels for language modeling, it is still useful to inspect the data at the start.

Now the data needs to be prepared for masked language modeling. This is a bit different from sequence classification.

Preprocessing the data

For both auto-regressive and masked language modeling, a common preprocessing step is:

1. concatenate all examples together
2. then split the whole corpus into equal-sized chunks

This is different from our previous approach, where we tokenized individual examples.

The reason for concatenating everything is that if an example is too long, tokenizing it individually may truncate it, and that can cause us to lose useful information.

First, we will tokenize the corpus, but without setting `truncation=True`.

If the tokenizer is fast, we will also keep `word_ids`, because we will need them later for whole word masking.

At the same time, we remove the `text` and `label` columns since we no longer need them.

```
def tokenize_function(examples):  
    result = tokenizer(examples["text"])
```

```
if tokenizer.is_fast:
    result["word_ids"] = [result.word_ids(i) for i in range(len(result["input_ids"]))]
return result
```

```
# Use batched=True to activate fast multithreading!
tokenized_datasets = imdb_dataset.map(
    tokenize_function, batched=True, remove_columns=["text", "label"]
)
tokenized_datasets
```

Output:

```
DatasetDict({
  train: Dataset({
    features: ['attention_mask', 'input_ids', 'word_ids'],
    num_rows: 25000
  })
  test: Dataset({
    features: ['attention_mask', 'input_ids', 'word_ids'],
    num_rows: 25000
  })
  unsupervised: Dataset({
    features: ['attention_mask', 'input_ids', 'word_ids'],
    num_rows: 50000
  })
})
```

Since DistilBERT is a BERT-like model, the encoded text contains:

- `input_ids`
- `attention_mask`
- `word_ids`

Now we need to group the tokenized reviews and split them into chunks. But how large should each chunk be?

That depends on GPU memory. A good place to start is to check the tokenizer's `model_max_length` to see the maximum context size.

```
tokenizer.model_max_length
```

Output:

512

So the model's context size is 512 tokens, just like BERT.

However, to make it fit into GPUs like the ones available in Colab, a smaller chunk size is used here:

```
chunk_size = 128
```

In real-world applications, a very small chunk size can be harmful, so you should choose a size that matches your actual use case.

Now we check the token lengths of a few tokenized reviews:

```
# Slicing produces a list of lists for each feature
tokenized_samples = tokenized_datasets["train"][:3]

for idx, sample in enumerate(tokenized_samples["input_ids"]):
    print(f">>> Review {idx} length: {len(sample)}")
```

Output:

```
'>>> Review 0 length: 200'
'>>> Review 1 length: 559'
'>>> Review 2 length: 192'
```

Then these are concatenated together:

```
concatenated_examples = {
    k: sum(tokenized_samples[k], []) for k in tokenized_samples.keys()
}
total_length = len(concatenated_examples["input_ids"])
print(f">>> Concatenated reviews length: {total_length}")
```

Output:

```
'>>> Concatenated reviews length: 951'
```

After that, the concatenated reviews are split according to `chunk_size`:

```
chunks = {
    k: [t[i : i + chunk_size] for i in range(0, total_length, chunk_size)]
    for k, t in concatenated_examples.items()
}

for chunk in chunks["input_ids"]:
    print(f">>> Chunk length: {len(chunk)}")
```

Output:

```
'>>> Chunk length: 128'
'>>> Chunk length: 128'
'>>> Chunk length: 128'
'>>> Chunk length: 128'
'>>> Chunk length: 128'
'>>> Chunk length: 128'
'>>> Chunk length: 128'
'>>> Chunk length: 55'
```

The last chunk is usually smaller. There are two ways to handle this:

- drop the chunk if it is smaller than `chunk_size`
- or pad it so its length becomes equal to `chunk_size`

Here, the first strategy is used. So all the logic is wrapped into one function called `group_texts()`:

```
def group_texts(examples):
    # Concatenate all texts
    concatenated_examples = {k: sum(examples[k], []) for k in examples.keys()}
    # Compute length of concatenated texts
    total_length = len(concatenated_examples[list(examples.keys())[0]])
    # We drop the last chunk if it's smaller than chunk_size
    total_length = (total_length // chunk_size) * chunk_size
    # Split by chunks of max_len
    result = {
        k: [t[i : i + chunk_size] for i in range(0, total_length, chunk_size)]
        for k, t in concatenated_examples.items()
    }
    # Create a new labels column
    result["labels"] = result["input_ids"].copy()
    return result
```

In the final step, a new column called `labels` is created, which is just a copy of `input_ids`. This is because in masked language modeling, the objective is to predict randomly masked tokens from the input batch, and the `labels` column provides the ground truth for the model.

Now this function is applied using `Dataset.map()`:

```
lm_datasets = tokenized_datasets.map(group_texts, batched=True)
lm_datasets
```

Output:

```
DatasetDict({
  train: Dataset({
    features: ['attention_mask', 'input_ids', 'labels', 'word_ids'],
    num_rows: 61289
  })
  test: Dataset({
    features: ['attention_mask', 'input_ids', 'labels', 'word_ids'],
    num_rows: 59905
  })
  unsupervised: Dataset({
    features: ['attention_mask', 'input_ids', 'labels', 'word_ids'],
    num_rows: 122963
  })
})
```

After grouping and chunking, the number of examples increases a lot. That is because we now create many new chunks of contiguous tokens, and those chunks may span across multiple original examples.

To understand this, one chunk is decoded:

```
tokenizer.decode(lm_datasets["train"][1]["input_ids"])
```

Here we can see `[SEP]` and `[CLS]` tokens, which means that parts of two different reviews are now inside the same chunk.

If we decode the `labels`:

```
tokenizer.decode(lm_datasets["train"][1]["labels"])
```

It looks exactly like `input_ids`.

So how will the model learn anything?

That is because one important step is still missing: inserting `[MASK]` tokens at random positions in the input. This will be done on the fly during fine-tuning using a special data collator.

Simple Explanation

Now let's understand it in a very easy way.

What is the main idea of this part?

Here, the author is showing how to prepare movie review data so that a language model can be trained on it.

Earlier, the model had learned from general text such as Wikipedia.

Now we want the model to learn the language of movie reviews, such as:

- “awesome”
- “boring”
- “waste of time”
- “great acting”
- “terrible plot”

That is why the IMDb dataset is being used.

What is inside the dataset?

The IMDb dataset has 3 parts:

- `train` → for training
- `test` → for testing
- `unsupervised` → extra text data without labels

And the labels are:

- `0` = negative
- `1` = positive

But here, our main goal is not sentiment classification.
We are fine-tuning a language model.
So the labels are not very important; the text is what matters most.

Why are the reviews not tokenized one by one?

Because some reviews can be very long.

If you tokenize each review separately and it goes beyond the maximum length, the rest gets cut off. That means important information is lost.

So what is being done instead?

All the review tokens are joined together first, and then they are cut into fixed-size chunks.

This is like having 3 paragraphs and, instead of reading them separately, joining them all together and then dividing them into blocks of 128 words.

That is what is happening here, except it is being done with tokens instead of words.

What do we get after tokenization?

`tokenize_function()` converts the text into the model's language.

Here we get:

- `input_ids` → the numeric IDs of the tokens
- `attention_mask` → which tokens are real and which are padding
- `word_ids` → which token belongs to which word; this helps later with whole-word masking

What does `chunk_size = 128` mean?

It means each final training example will contain 128 tokens.

For example:

- first 128 tokens = chunk 1

- next 128 tokens = chunk 2
- and so on

If 55 tokens remain at the end:

- they can be dropped, or
- they can be padded

Here, they are dropped.

What does the `group_texts()` function do?

This function does 4 things:

1. Joins all tokens together

```
concatenated_examples = {k: sum(examples[k], []) for k in examples.keys()}
```

2. Finds the total length

```
total_length = len(...)
```

3. Cuts the sequence according to `chunk_size`

```
[t[i : i + chunk_size] for i in range(0, total_length, chunk_size)]
```

4. Creates `labels`

```
result["labels"] = result["input_ids"].copy()
```

Why is `labels = input_ids.copy()`?

This is where many people get confused.

In masked language modeling, the model's job is to predict the masked word.

Suppose the input is:

`This movie is [MASK].`

And the original sentence was:

`This movie is amazing.`

Then:

- the input will contain `[MASK]`
- the label will contain the original token: `amazing`

At this point, masking has not been applied yet.

So for now, `labels` and `input_ids` are kept the same.

Later, random masks will be inserted.

So is the model not learning yet?

Exactly — the full learning setup is not complete yet.

Because:

- right now the input and label are the same
- there are no `[MASK]` tokens yet

In the next step, a data collator will randomly mask some tokens.

Then the model will learn:

“What word was originally in this masked position?”

Why did the number of examples increase so much?

Earlier, the train split had 25,000 reviews.

After processing, it became 61,289 chunks.

That is because one long review can produce multiple chunks.

Also, one chunk can contain pieces of two different reviews.

Why do we see `[SEP]` and `[CLS]`?

BERT-type models use special tokens.

- [CLS] → marks the beginning of a sentence or chunk
- [SEP] → indicates a boundary

When the chunk was decoded, it became clear that one chunk may now contain parts of multiple reviews.

Fine-tuning DistilBERT with the Trainer API

Fine-tuning a masked language model is almost the same as fine-tuning a sequence classification model, like we did in Chapter 3.

The main difference is that here we need a special **data collator** that can randomly mask some of the tokens in each batch of text.

Fortunately, Transformers already provides a separate **DataCollatorForLanguageModeling** for this purpose.

We only need to pass the tokenizer and `mlm_probability`, which specifies what fraction of the total tokens will be masked.

Here we will use **15%**, because that is the amount used in BERT and is also a common choice in the literature.

```
from transformers import DataCollatorForLanguageModeling
```

```
data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm_probability=0.15)
```

To see how random masking works, a few examples are passed to the data collator.

Since the collator expects a list of dictionaries, where each dictionary represents one chunk of contiguous text, we first take samples from the dataset.

Here, the `"word_ids"` key is removed, because this data collator does not expect it.

```
samples = [lm_datasets["train"][i] for i in range(2)]
```

```
for sample in samples:
```

```
    _ = sample.pop("word_ids")
```

```
for chunk in data_collator(samples)["input_ids"]:
```

```
    print(f"\n>>> {tokenizer.decode(chunk)}")
```

In the output, we can see that random `[ MASK ]` tokens have been inserted in different places.

During training, the model will have to predict these masked tokens.

The nice thing about the data collator is that the masking positions will be randomized again for every batch.

One side effect is that if the same data collator is used for both train and test, the evaluation metric will not be deterministic, because the masking changes every time.

Later, with Accelerate, this randomness can be frozen by building a custom evaluation loop.

Whole word masking

Another technique in masked language modeling is to mask whole words together, instead of masking only individual tokens.

This is called **whole word masking**.

If we want to use it, we need to build our own data collator.

A data collator is basically a function that takes a list of samples and turns them into a batch.

Here, the previously computed `word_ids` are used to create a mapping between word indices and their corresponding tokens.

Then some words are randomly selected for masking.

In this case, all positions in `labels` will contain `-100` except the ones corresponding to the masked words.

```
import collections
import numpy as np
```

```
from transformers import default_data_collator
```

```
wwm_probability = 0.2
```

```
def whole_word_masking_data_collator(features):
    for feature in features:
        word_ids = feature.pop("word_ids")

        # Create a map between words and corresponding token indices
        mapping = collections.defaultdict(list)
        current_word_index = -1
        current_word = None
        for idx, word_id in enumerate(word_ids):
            if word_id is not None:
                if word_id != current_word:
                    current_word = word_id
                    current_word_index += 1
                mapping[current_word_index].append(idx)

        # Randomly mask words
        mask = np.random.binomial(1, wwm_probability, (len(mapping),))
        input_ids = feature["input_ids"]
        labels = feature["labels"]
        new_labels = [-100] * len(labels)
        for word_id in np.where(mask)[0]:
            word_id = word_id.item()
            for idx in mapping[word_id]:
                new_labels[idx] = labels[idx]
```

```

        input_ids[idx] = tokenizer.mask_token_id
        feature["labels"] = new_labels

    return default_data_collator(features)

```

Now this can be applied to the same samples as before:

```

samples = [lm_datasets["train"][i] for i in range(2)]
batch = whole_word_masking_data_collator(samples)

for chunk in batch["input_ids"]:
    print(f"\n'>>> {tokenizer.decode(chunk)}")

```

In the output, we can see that all tokens belonging to the same word are masked together.

Reducing the dataset size for training

Since the remaining fine-tuning steps are standard, and training can take a long time on Google Colab, the training set is reduced in size.

A fairly good language model can still be obtained with just a few thousand examples.

This can be done quickly using the `Dataset.train_test_split()` function from Datasets.

```

train_size = 10_000
test_size = int(0.1 * train_size)

downsampled_dataset = lm_datasets["train"].train_test_split(
    train_size=train_size, test_size=test_size, seed=42
)
downsampled_dataset

```

Output:

```

DatasetDict({
  train: Dataset({
    features: ['attention_mask', 'input_ids', 'labels', 'word_ids'],
    num_rows: 10000
  })
  test: Dataset({
    features: ['attention_mask', 'input_ids', 'labels', 'word_ids'],
    num_rows: 1000
  })
})

```

```
    })  
  })
```

Here:

- training set = 10,000 examples
- validation/test set = 1,000 examples

If the GPU is stronger, this can be increased further.

Logging in to the Hugging Face Hub

Next, we need to log in to the Hugging Face Hub. If using a notebook:

```
from huggingface_hub import notebook_login  
  
notebook_login()
```

This will show a widget where credentials can be entered.
Or, in the terminal, you can run:

```
huggingface-cli login
```

Setting TrainingArguments

Now the arguments for the `Trainer` are defined:

```
from transformers import TrainingArguments  
  
batch_size = 64  
# Show the training loss with every epoch  
logging_steps = len(downsampled_dataset["train"]) // batch_size  
model_name = model_checkpoint.split("/")[-1]  
  
training_args = TrainingArguments(  
    output_dir=f"{model_name}-finetuned-imdb",  
    overwrite_output_dir=True,  
    evaluation_strategy="epoch",
```

```

learning_rate=2e-5,
weight_decay=0.01,
per_device_train_batch_size=batch_size,
per_device_eval_batch_size=batch_size,
push_to_hub=True,
fp16=True,
logging_steps=logging_steps,
)

```

Here, some important options were changed:

- `evaluation_strategy="epoch"` → evaluation after each epoch
- `learning_rate=2e-5`
- `weight_decay=0.01`
- `batch_size=64`
- `push_to_hub=True` → the trained model will be pushed to the Hub
- `fp16=True` → mixed-precision training, which increases speed

Another important point:

By default, `Trainer` removes any columns that are not part of the model's `forward()` method.

So if you use the whole word masking collator, you must set `remove_unused_columns=False`, otherwise the `word_ids` column will be lost.

Also, with the `hub_model_id` argument, you can manually specify which repository on the Hub the model should be pushed to.

If you do not specify it, then by default a repository will be created in your own namespace using the name of `output_dir`.

Creating the Trainer

Now that all the ingredients are ready, the `Trainer` is instantiated.

Here the standard `data_collator` is used, but as an exercise you can also try the whole word masking collator.

```

from transformers import Trainer

```

```

trainer = Trainer(
    model=model,
    args=training_args,

```

```
train_dataset=downsampled_dataset["train"],
eval_dataset=downsampled_dataset["test"],
data_collator=data_collator,
tokenizer=tokenizer,
)
```

Now everything is ready to run `trainer.train()`.

But before that, the text briefly discusses a common metric for measuring language model performance: **perplexity**.

Simple Explanation

Now let's understand it very easily.

What is the main idea of this part?

In the previous part, we prepared the data.

This part shows:

how the actual training will begin.

That means:

- inserting random masks
- creating batches
- setting up training
- preparing fine-tuning with the Trainer

What is a data collator?

A data collator is a helper that prepares batches right before training.

Its job is:

- to combine samples into a batch
- to add padding if needed
- and here, as an extra step, to insert random `[MASK]` tokens

What does `DataCollatorForLanguageModeling` do?

It automatically masks some tokens at random.

```
data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm_probability=0.15)
```

Here, `mlm_probability=0.15` means:

- 15% of the total tokens will be masked

Suppose there are 100 tokens.

Then about 15 of them will become `[MASK]`.

Why is masking necessary?

Because the job of a masked language model is to predict missing words.

For example:

The movie was `[MASK]`.

The model will guess:

- amazing
- terrible
- boring
- fantastic

So without `[MASK]`, the model would not have any actual task to perform.

Why does random masking change in every batch?

That is the powerful part of this method.

The same sentence may be masked in one position today, and in another position in the next batch.

So the model can keep learning from the same sentence in new ways.

Why was `word_ids` removed with `pop()`?

Because the normal `DataCollatorForLanguageModeling` expects only standard input. It does not work with `word_ids`.

So:

```
_ = sample.pop("word_ids")
```

is used to remove `word_ids`.

What is whole word masking?

In ordinary masking, only part of a word may be masked.

Suppose the word is `playing`.

After tokenization, it might become:

- `play`
- `##ing`

With ordinary masking, only `play` might get masked while `##ing` stays visible.

But in **whole word masking**:

- all tokens of the whole word are masked together

How does the whole word masking collator work?

This custom function does the following:

1. Uses `word_ids` to find which token belongs to which word
2. Randomly chooses some words
3. Replaces all tokens of those words with `[MASK]`

4. Keeps the original token values only in the masked positions inside **labels**

5. Fills the rest of **labels** with **-100**

Why is **-100** used?

This is very important.

In PyTorch loss functions, **-100** means:

ignore this position

That means the model calculates loss only for the masked positions.
The other positions do not contribute to the loss.

Why was the training set reduced?

Because training on the full dataset can be very slow, especially in Colab.

So they used:

- 10,000 training examples
- 1,000 validation examples

This makes training faster while still producing a decent model.

Why is Hugging Face login needed?

Because the trained model will be uploaded to the Hub.

Since **push_to_hub=True** is used, logging in is required.

What are the important parts of **TrainingArguments**?

output_dir

Where the trained model will be saved

evaluation_strategy="epoch"

Evaluation will happen after each epoch

learning_rate=2e-5

How fast the model will learn

weight_decay=0.01

Helps reduce overfitting

per_device_train_batch_size=64

How many samples go to the GPU at one time

fp16=True

Uses half precision to speed up training

push_to_hub=True

Uploads the model to the Hugging Face Hub after training

Why might `remove_unused_columns=False` be needed?

The whole word masking collator uses `word_ids`.

But `Trainer` removes unused columns by default.

So if you use custom whole word masking and want to keep `word_ids`, you need:

```
remove_unused_columns=False
```

What is the **Trainer** object?

Trainer is Hugging Face's ready-made training manager.

It can:

- run the training loop
- evaluate
- log progress
- save checkpoints
- push to the Hub

In this code:

```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=downsampled_dataset["train"],  
    eval_dataset=downsampled_dataset["test"],  
    data_collator=data_collator,  
    tokenizer=tokenizer,  
)
```

This means the Trainer is being told:

- which model to train
- which dataset to use
- which settings to follow
- how masking should be done

Code flow in a very simple way

The full flow is:

Step 1:

Create a data collator

→ it will insert random masks

Step 2:

Optionally create a custom whole word masking collator

Step 3:

Reduce the training dataset

→ training becomes faster

Step 4:

Log in to the Hugging Face Hub

Step 5:

Set `TrainingArguments`

Step 6:

Create the `Trainer`

Step 7:

Run `trainer.train()`

Perplexity for language models

In tasks like text classification or question answering, we have a labeled corpus. But in language modeling, we do not have such explicit labels.

So how do we know whether a language model is good or not?

Think of it like the autocorrect feature on your phone:

A good language model is one that assigns high probability to grammatically correct sentences, and low probability to strange or nonsense sentences.

There are many “autocorrect fail” examples online, where a phone model produced funny or inappropriate completions.

If most of the sentences in our test set are grammatically correct, then one way to measure the quality of a language model is to calculate the probabilities the model assigns to the next word in the sentences of the test set.

If the probability is high, that means the model is not very “surprised” or “perplexed” by unseen examples.

That suggests the model has learned the basic patterns of grammar in the language.

There are different mathematical definitions of perplexity, but here we use the one defined as the **exponential of the cross-entropy loss**.

So to compute the perplexity of the pretrained model, we use `Trainer.evaluate()` to compute the cross-entropy loss on the test set, and then take its exponential.

```
import math
```

```
eval_results = trainer.evaluate()  
print(f">>> Perplexity: {math.exp(eval_results['eval_loss']):.2f}")
```

Output:

Perplexity: 21.75

A lower perplexity means a better language model.

Here the initial score looks somewhat high.

Now let's see whether we can reduce it through fine-tuning. First, the training loop is run:

```
trainer.train()
```

Then perplexity is computed again in the same way:

```
eval_results = trainer.evaluate()  
print(f">>> Perplexity: {math.exp(eval_results['eval_loss']):.2f}")
```

Output:

Perplexity: 11.32

This has gone down quite a lot, which shows that the model has learned something about the domain of movie reviews.

After training is finished, the training information and model card can be pushed to the Hub:

```
trainer.push_to_hub()
```

Fine-tuning DistilBERT with Accelerate

As we saw with Trainer, fine-tuning a masked language model is very similar to the text classification example from Chapter 3.

The only subtle difference is the use of a special data collator, which we already discussed earlier.

However, we also saw that `DataCollatorForLanguageModeling` applies random masking during evaluation as well.

As a result, the perplexity score may fluctuate a bit in each training run.

One way to reduce this randomness is to apply masking once to the test set, and then use the default data collator during evaluation.

To demonstrate this, a function is created that applies masking to a batch:

```
def insert_random_mask(batch):
    features = [dict(zip(batch, t)) for t in zip(*batch.values())]
    masked_inputs = data_collator(features)
    # Create a new "masked" column for each column in the dataset
    return {"masked_" + k: v.numpy() for k, v in masked_inputs.items()}
```

Then this function is applied to the test set, and the unmasked columns are dropped while the masked columns are renamed:

```
downsampled_dataset = downsampled_dataset.remove_columns(["word_ids"])
eval_dataset = downsampled_dataset["test"].map(
    insert_random_mask,
    batched=True,
    remove_columns=downsampled_dataset["test"].column_names,
)
```

```
eval_dataset = eval_dataset.rename_columns(
    {
        "masked_input_ids": "input_ids",
        "masked_attention_mask": "attention_mask",
        "masked_labels": "labels",
    }
)
```

Now the dataloaders can be set up in the usual way, but for the evaluation set we will use `default_data_collator`:

```
from torch.utils.data import DataLoader
from transformers import default_data_collator

batch_size = 64
train_dataloader = DataLoader(
    downsampled_dataset["train"],
    shuffle=True,
    batch_size=batch_size,
    collate_fn=default_data_collator,
)
eval_dataloader = DataLoader(
    eval_dataset, batch_size=batch_size, collate_fn=default_data_collator
)
```

After that, the standard steps of Accelerate are followed.
First, a fresh version of the pretrained model is loaded:

```
model = AutoModelForMaskedLM.from_pretrained(model_checkpoint)
```

Then the optimizer is defined; here the standard AdamW is used:

```
from torch.optim import AdamW

optimizer = AdamW(model.parameters(), lr=5e-5)
```

Next, everything is prepared for training using the `Accelerator` object:

```
from accelerate import Accelerator

accelerator = Accelerator()
```

```
model, optimizer, train_dataloader, eval_dataloader = accelerator.prepare(
    model, optimizer, train_dataloader, eval_dataloader
)
```

Now the learning rate scheduler is defined:

```
from transformers import get_scheduler
```

```
num_train_epochs = 3
num_update_steps_per_epoch = len(train_dataloader)
num_training_steps = num_train_epochs * num_update_steps_per_epoch
```

```
lr_scheduler = get_scheduler(
    "linear",
    optimizer=optimizer,
    num_warmup_steps=0,
    num_training_steps=num_training_steps,
)
```

Before training begins, a model repository must be created on the Hugging Face Hub. First, the full name of the repo is generated:

```
from huggingface_hub import get_full_repo_name

model_name = "distilbert-base-uncased-finetuned-imdb-accelerate"
repo_name = get_full_repo_name(model_name)
repo_name
```

Output:

```
'lewtun/distilbert-base-uncased-finetuned-imdb-accelerate'
```

Then the **Repository** class is used to create and clone the repo:

```
from huggingface_hub import Repository

output_dir = model_name
repo = Repository(output_dir, clone_from=repo_name)
```

After that, the full training and evaluation loop is written manually:

```

from tqdm.auto import tqdm
import torch
import math

progress_bar = tqdm(range(num_training_steps))

for epoch in range(num_train_epochs):
    # Training
    model.train()
    for batch in train_dataloader:
        outputs = model(**batch)
        loss = outputs.loss
        accelerator.backward(loss)

        optimizer.step()
        lr_scheduler.step()
        optimizer.zero_grad()
        progress_bar.update(1)

    # Evaluation
    model.eval()
    losses = []
    for step, batch in enumerate(eval_dataloader):
        with torch.no_grad():
            outputs = model(**batch)

        loss = outputs.loss
        losses.append(accelerator.gather(loss.repeat(batch_size)))

    losses = torch.cat(losses)
    losses = losses[: len(eval_dataset)]
    try:
        perplexity = math.exp(torch.mean(losses))
    except OverflowError:
        perplexity = float("inf")

    print(f">>> Epoch {epoch}: Perplexity: {perplexity}")

    # Save and upload
    accelerator.wait_for_everyone()
    unwrapped_model = accelerator.unwrap_model(model)
    unwrapped_model.save_pretrained(output_dir, save_function=accelerator.save)
    if accelerator.is_main_process:
        tokenizer.save_pretrained(output_dir)

```

```
repo.push_to_hub(  
    commit_message=f"Training in progress epoch {epoch}", blocking=False  
)
```

Output:

```
Epoch 0: Perplexity: 11.397545307900472  
Epoch 1: Perplexity: 10.904909330983092  
Epoch 2: Perplexity: 10.729503505340409
```

Here we can see that perplexity decreases with each epoch, and multiple training runs become reproducible.

Using the fine-tuned model

The fine-tuned model can be used either through the widget on the Hub or locally using `pipeline`.

Here, the model is loaded using the `fill-mask` pipeline:

```
from transformers import pipeline  
  
mask_filler = pipeline(  
    "fill-mask", model="huggingface-course/distilbert-base-uncased-finetuned-imdb"  
)
```

Then the previous sample text, `"This is a great [MASK]"`, is passed to the model:

```
preds = mask_filler(text)  
  
for pred in preds:  
    print(f">>> {pred['sequence']}")
```

Output:

```
'>>> this is a great movie.'  
'>>> this is a great film.'  
'>>> this is a great story.'  
'>>> this is a great movies.'  
'>>> this is a great character.'
```

From this, we can see that the model now predicts movie-related words, which means it has adapted to the domain.

This is how the first experiment with language model training comes to an end.
In Section 6, you will learn how to train an auto-regressive model like GPT-2 from scratch.

Simple Explanation

Now let's understand it in a very easy way.

What is the main idea of this part?

This part has 3 big ideas:

- measuring whether the model improved using **perplexity**
- training with **Trainer**
- if needed, writing a custom training loop with more control using **Accelerate**
- and then using the fine-tuned model to see predictions

What is Perplexity?

Perplexity is the confusion score of a language model.

Simply put:

- **lower perplexity = less confused model = better model**
- **higher perplexity = more confused model = worse model**

Suppose the model looks at a sentence and feels:
“What could the next word possibly be?”

Then perplexity will be high.

But if the model can predict the next word confidently, then perplexity will be low.

Why do we need perplexity?

In language modeling, there is no class label.

For example, in sentiment analysis we have:

- positive
- negative

But here, the model's job is to predict the next token or the masked token.
So accuracy is not always the easiest metric to use.

That is why perplexity is used.

Why `math.exp(eval_loss)`?

Because here perplexity is defined as:

perplexity = $\exp(\text{cross-entropy loss})$

So:

- first `trainer.evaluate()` gives `eval_loss`
- then `math.exp()` is used to take the exponential

What does 21.75 before training and 11.32 after training mean?

This is very important.

Before training:

- perplexity = **21.75**

After training:

- perplexity = **11.32**

That means the model is much less confused than before.

In other words:

- it has learned the movie review domain
- it now understands words and contexts like “movie,” “film,” and “story” better

What does `trainer.train()` actually do?

It runs the actual training loop.

Here the model:

- takes batches
- predicts masked words
- calculates loss
- updates weights
- gradually becomes better

What does `trainer.push_to_hub()` do?

It uploads the trained model to the Hugging Face Hub.

As a result:

- it can be loaded again later
- others can use it
- the model card is also saved

Then why use `Accelerate`?

Because `Trainer` is simple, but it gives limited control.

With `Accelerate`, you can:

- write the training loop yourself
- customize evaluation
- control randomness
- more easily use multi-GPU or mixed precision

Why is random masking a problem during evaluation?

The earlier data collator also inserts random masks during evaluation.

So:

- today one part of the test set is masked
- tomorrow another part is masked

That means the perplexity score may change a little.
So the result is not fully deterministic.

What is the solution?

Apply masking to the test set only once.

Then use that fixed masked version during evaluation.

This makes:

- the evaluation set fixed
- the results stable
- perplexity consistent from run to run

What does `insert_random_mask()` do?

This function:

- takes a batch
- inserts masks using the data collator
- returns the masked version

So it prepares a pre-masked test set for evaluation.

Why are there two different dataloaders?

`train_dataloader`

Random masking is used here

so `collate_fn=data_collator`

`eval_dataloader`

A fixed masked dataset is used here

so `collate_fn=default_data_collator`

This makes training random but evaluation stable.

What does **Accelerator** do?

```
accelerator = Accelerator()
```

```
model, optimizer, train_dataloader, eval_dataloader = accelerator.prepare(...)
```

It makes the training setup device-ready.

That means:

- it handles CPU/GPU
- it supports multi-GPU
- it supports mixed precision
- it makes distributed setup easier

What is AdamW?

It is the optimizer.

The optimizer's job is:

- to look at the loss
- update the model's weights
- so that the model learns better

AdamW is a very common choice in Transformer training.

What does the learning rate scheduler do?

During training, a scheduler is used to gradually reduce or control the learning rate.

Here:

```
get_scheduler("linear", ...)
```

means the learning rate will be adjusted linearly.

The training loop in very simple terms

Each epoch has 2 phases

1) Training

- `model.train()`
- takes a batch
- computes loss
- does backpropagation
- optimizer takes a step
- learning rate is updated

2) Evaluation

- `model.eval()`
- gradients are turned off
- losses are collected
- perplexity is computed from the mean loss

Then:

- the model is saved
- it is pushed to the Hub

Why `accelerator.backward(loss)`?

This is an Accelerate-friendly version of normal `loss.backward()`.

It is safer and more useful for distributed or mixed precision training.

Why `accelerator.gather(...)`?

If there are multiple devices, the loss from all devices must be gathered together.

That ensures the final perplexity is calculated correctly.

What does it mean that perplexity decreases by epoch?

Epoch 0: 11.39

Epoch 1: 10.90

Epoch 2: 10.73

This is a healthy sign.

It means:

- the model is improving in every epoch
- it is learning the movie review domain better

What happened when the fine-tuned model was used?

Earlier, the generic model predicted common words after “great,” such as:

- success
- idea
- feat

But the fine-tuned model predicted:

- movie
- film
- story
- character

That means the model has now picked up the style of the movie review domain.
That is the success of domain adaptation.

A very simple analogy

Suppose a student first learned general English.

Then you made that student read many movie reviews.

Now you say:

“This is a great ____”

Now the student will say “movie” or “film.”

Earlier they might have said “idea” or “success.”

That is the effect of fine-tuning.